

Evidence-Bounded Structural Reproduction of a GPU sGS-HPR Solver for Large-Scale Multi-Period DC Optimal Power Flow

Thomas DiGregorio

Abstract—We independently reconstruct a published GPU symmetric Gauss–Seidel Halpern-accelerated proximal-reflection method for multi-period DC optimal power flow without the authors’ code or instances. The canonical LP, iteration, stopping maps, structural equality solve, CPU oracle, and resident DGX Spark GPU implementation are validated. A sign inconsistency in the printed rank-one inverse is corrected: maximum relative error falls to 3.03×10^{-16} from median error 0.206. Public MATPOWER models reproduce all 18 reported dimensions but none of the nonzero counts. The GPU validates 11 allocated cases and the CPU 10; larger cases meet time, memory, or sparse-index limits. These results establish structural—not exact or performance—reproduction; no A100-equivalent speedup is claimed.

Index Terms—DC optimal power flow, Halpern iteration, proximal reflection, symmetric Gauss–Seidel decomposition, GPU sparse linear algebra, reproducibility, DGX Spark.

I. INTRODUCTION

Wang *et al.* propose a GPU sGS-HPR method for large-scale multi-period DC optimal power flow (DCOPF) and report results on an A100-SXM4-80GB GPU [1]. Reproduction requires separating the mathematical map, numerical instance, implementation, validation criteria, timer boundary, and hardware. This study asks which layers can be recovered from the paper and public artifacts.

The mathematical path and an FP64 GPU implementation are independently validated, including a correction to the structural equality solve. Public models reproduce the published dimensions but not the sparse supports, and the local hardware and timers differ from the source study. We therefore use *structural reproduction* to mean that the algorithm, implementation structure, public-network scaling, and validation behavior are reproduced without claiming author-instance identity or controlled paper timing.

The contributions are:

- a self-contained canonical DCOPF and sGS-HPR implementation contract;
- independent residual, physical, direct-solve, and CPU/GPU trajectory checks;

Independent Researcher. Correspondence, code, data, and the immutable report release are available at <https://github.com/thomas-digregorio/sGS-HPR-DC-SCOPF>.

- a formal correction and stable implementation of the structural equality inverse;
- an 18-case structural ledger, an 11-case benchmark record, and repeated timing distributions; and
- live-memory and sparse-index boundary evidence with a public, versioned code/data release.

The displayed source model has no outage or contingency index. This report therefore studies base-case multi-period DCOPF; N-1 contingency analysis is outside the scope of this study.

II. METHODS

A. Source paper, scope, and public data

The source paper is the 17-page article by Wang *et al.* [1]. It applies HPR to the dual of a box-constrained LP, splits equality and inequality multipliers with an sGS sweep, and uses CSR sparse products on an A100. Its $O(1/k)$ KKT residual result follows the HPR framework [2].

Public base networks are MATPOWER 8.1 case1354pegase, case2868rte, and case9241pegase [3], [4]. The PEGASE/RTE provenance is described in [5], [4]. The authors’ renewable/storage placements, time series, reserve profiles, modified ramps, storage data, matrix construction, exact control code, and timer boundary were unavailable. Appendix A gives the deterministic replacement policy.

B. Mathematical formulation

The implementation contract is

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & c^\top x, \\ \text{s.t.} \quad & A_1 x = b_1, \\ & A_2 x \geq b_2, \\ & \ell \leq x \leq u, \end{aligned} \tag{1}$$

where $A = [A_1; A_2]$, $b = [b_1; b_2]$, and $C = [\ell, u]$. With $D = \mathbb{R}^{m_1} \times \mathbb{R}_+^{m_2}$, the dual is

$$\begin{aligned} \min_{y, z} \quad & -b^\top y + \delta_D(y) + \delta_C^*(-z), \\ \text{s.t.} \quad & A^\top y + z = c. \end{aligned} \tag{2}$$

Equality multipliers are free; inequality multipliers are nonnegative. The canonical inequality direction is always $A_2 x \geq b_2$.

The decision order is

$$x = (p_G, p_{RG}, p_{ESS}^{dc}, p_{ESS}^{ch}, r^u, r^d). \tag{3}$$

For T periods, N_L constrained lines, N_G conventional generators, N_{RG} renewables, and N_{ESS} storage devices,

$$n = T(3N_G + N_{RG} + 2N_{ESS}), \quad (4)$$

$$m_1 = T + N_{ESS}, \quad (5)$$

$$m_2 = 2TN_L + (4T - 2)N_G + 2TN_{ESS} + 2T. \quad (6)$$

Power balance and terminal storage energy form A_1 . Split branch flow, headroom/footroom, reserve, ramp, and storage-energy rows form A_2 ; remaining device limits form C .

The full KKT residual is

$$\mathcal{R}(y, z, x) = \begin{pmatrix} y - \Pi_D(y - Ax + b) \\ x - \Pi_C(x - z) \\ c - A^\top y - z \end{pmatrix}. \quad (7)$$

The paper's stopping rule separately normalizes primal feasibility, box normal-cone consistency, and stationarity. Its first stopping block is not the entire first block of (7); both maps are retained.

C. Implemented algorithm and sourced control policy

For $w = (y_1, y_2, z, x)$, the FP64 implementation follows the printed update order. The combined z /primal update gives

$$\bar{x}^{k+1} = \Pi_C[x^k + \sigma(A^\top y^k - c)]. \quad (8)$$

An equality solve computes $\bar{y}_1^{k+1/2}$ using y_2^k . Then

$$\bar{y}_2^{k+1} = \Pi_{\mathbb{R}_+^{m_2}}[y_2^k + \lambda^{-1}(b_2/\sigma - A_2 R_y)], \quad (9)$$

where λ safely bounds $\|A_2\|_2^2$. A second equality solve uses the new $\bar{y}_2^{k+1}$ to obtain $\bar{y}_1^{k+1}$. Finally,

$$\hat{w}^{k+1} = 2\bar{w}^{k+1} - w^k, \quad (10)$$

$$w^{k+1} = \frac{1}{k+2}w^0 + \frac{k+1}{k+2}\hat{w}^{k+1}. \quad (11)$$

The anchor w^0 is fixed.

Production preprocessing applies ten Ruiz equilibration passes [6], one simultaneous Pock–Chambolle diagonal step with $\alpha = 1$ [7], and complete-vector b/c normalization. Initial $\sigma = 1$. The DCOPF paper does not publish exact restart or adaptive-penalty code; Appendix C states the transferred HPR-LP rules, formulas, guards, and source pin explicitly [2]. This is a sourced reconstruction, not author-code identity.

D. Acceptance and timing protocol

Three independent tracks are used: HiGHS supplies an objective reference, the CPU implementation is an algorithmic oracle, and the GPU implementation is the target path. Their implementations and exact clock boundaries are documented in Appendix B; Table I states the frozen decision rules applied to all tracks.

III. VERIFICATION

A. Structural equality correction

Each sGS-HPR iteration solves an equality system involving $A_1 A_1^\top$. Its first T rows are hourly power balances

TABLE I
FROZEN ACCEPTANCE AND TIMING PROTOCOL.

Element	Frozen design
Numerical acceptance	Normalized stopping blocks $\leq 5e-5$; raw KKT ≤ 0.01 ; scaled objective gap to HiGHS $\leq 2e-4$.
Physical acceptance	Maximum original-unit violation ≤ 0.01 MW/MWh.
Row acceptance	HiGHS, CPU FP64, and GPU FP64 must each pass; a preallocation resource stop cannot be reclassified as a solve.
Correctness/warm-up	One untimed correctness solve, followed by one warm-up.
Measured repeats	Five measured solves; relative range above 0.2 triggers four additional repeats without deleting earlier samples.
Reported statistics	Median, minimum, maximum, standard deviation, and IQR.
Time limit	3,600 s per solve; a limit event is censored and has no median.
Clock interpretation	Solver-specific boundaries are reported separately and are not divided into cross-solver speedups.

and its remaining rows enforce terminal storage energy. Every balance row has the same structural weight, while each terminal-energy row couples to every period in the same way. The resulting Gram matrix is diagonal except for one direction shared by all periods. Lemma 1 exploits that structure instead of forming or factorizing a dense matrix.

Lemma 1 (stable inverse of the reduced equality block). Let $D_2 \succ 0$ be diagonal, $a > 0$, d a vector, and $\alpha = d^\top D_2^{-1} d$. For

$$\begin{bmatrix} aI_T & \mathbf{1}d^\top \\ d\mathbf{1}^\top & D_2 \end{bmatrix} \begin{bmatrix} y_{11} \\ y_{12} \end{bmatrix} = \begin{bmatrix} r_{11} \\ r_{12} \end{bmatrix}, \quad (12)$$

define

$$\hat{r} = r_{11} - \mathbf{1}d^\top D_2^{-1} r_{12}, \quad (13)$$

$$\gamma = a - T\alpha, \quad \mu = T^{-1}\mathbf{1}^\top \hat{r}. \quad (14)$$

If $\gamma > 0$, the unique solution is

$$y_{11} = \frac{\hat{r} - \mu\mathbf{1}}{a} + \frac{\mu}{\gamma}\mathbf{1}, \quad (15)$$

$$y_{12} = D_2^{-1}(r_{12} - d\mathbf{1}^\top y_{11}). \quad (16)$$

Interpretation. The vector $\mathbf{1}$ is the common-across-time direction. The adjusted right-hand side $\hat{r}$ separates into its average $\mu\mathbf{1}$ and a zero-mean deviation $\hat{r} - \mu\mathbf{1}$. The rank-one storage coupling does not act on the zero-mean part, so that part is divided by a . It acts only on the average, which is divided by the corrected scalar γ . The storage multipliers then follow from a diagonal back-substitution. Thus an apparently dense $(T + N_{ESS})$ -row solve requires only vector sums, diagonal divisions, and scalar operations.

Proof. Eliminating y_{12} yields $(aI_T - \alpha\mathbf{1}\mathbf{1}^\top)y_{11} = \hat{r}$. The zero-mean subspace has eigenvalue a and $\text{span}\{\mathbf{1}\}$ has eigenvalue $a - T\alpha = \gamma$. Decomposing $\hat{r}$ into mean and zero-mean components gives (15); back-substitution gives (16). $\square$

The printed expression uses the sign pattern of a plus rank-one update and fails a direct linear-system oracle. Equation (15) avoids forming the cancellation-prone Woodbury denominator in the solve. The code also evaluates γ through a positive-sum identity (Appendix D).

TABLE II
FROZEN GATES AND LARGEST ACCEPTED OBSERVATIONS.

Gate	Threshold	Maximum	Result
Normalized primal	5e−5	2.017e−8	Pass
Normalized stationarity	5e−5	9.659e−6	Pass
Normalized box	5e−5	8.831e−13	Pass
Raw KKT	0.01	0.009635	Pass
Physical violation	0.01	0.006311	Pass
Objective gap	2e−4	4.285e−8	Pass
Per-solve deadline	3600 s	3600.093 s	T6 CPU fail

TABLE III
STRUCTURAL-SOLVE ACCURACY AGAINST THE DIRECT FP64 ORACLE;
ENTRIES ARE MAXIMUM ERRORS OVER TESTED RIGHT-HAND SIDES.

Fixture	Rows	κ_2	Rel. error	Norm. residual
No storage, T1	1	1.00	2.22e−16	1.67e−16
No storage, T17	17	1.00	0	0
Ideal storage, T1	2	86.01	2.18e−16	6.66e−16
One storage, T2	3	3.58	3.03e−16	3.21e−16
Extreme efficiency, T32	33	25,616	3.00e−15	2.93e−14
Heterogeneous, T5	9	548.59	1.15e−15	1.05e−15
Many ideal, T16	48	292.03	1.53e−14	6.44e−15

On the sign fixture, the printed formula’s median relative error is 0.206; the corrected maximum relative error is 3.03×10^{-16} . Across condition numbers from 1 to 25,616, the largest normalized residual is 2.93×10^{-14} .

B. Numerical, physical, and trajectory checks

Analytic LPs verify box and nonnegative projections, (7), the separate stopping blocks, and the combined z /primal identity. Dense eigendecomposition is the small-case authority for λ ; sparse `eigsh` and seeded power iteration cross-check it. PTDF flows are compared with an independent angle solve, including phase-shift offsets. CPU/GPU checks compare intermediate trajectories, policy events, final objectives, and recovered physical states.

IV. RESULTS

A. Structural reconciliation

All 18 Table II (m, n) pairs match. Zero reconstructed nnz counts match; differences range from approximately −8.14% to −36.66%. Thus the public models reproduce the dimension algebra but not the author sparse supports. Paper/local timing ratios are invalid before hardware differences are considered.

B. Structural-solve accuracy

The corrected reduced inverse was compared with direct FP64 solves across seven fixtures spanning one to 48 equality rows and condition numbers up to 25,616. Table III reports errors in the original coordinates. The largest normalized residual is 2.93×10^{-14} ; the largest componentwise error is 4.42×10^{-9} .

C. Benchmark results

The GPU completed and validated all 11 allocated rows; CPU completed 10. Because acceptance requires all three solvers, T6 remains a failed row despite its valid HiGHS and GPU results.

TABLE IV
ALLOCATED PUBLIC-NETWORK RECONSTRUCTIONS.

Reconstruction	m	n	nnz(A)	Result
case1354 T4	20,192	4,208	4,799,808	Pass
case1354 T16	82,124	16,832	19,228,464	Pass
case1354 T48	247,276	50,496	57,896,368	Pass
case1354 T96	495,004	100,992	116,420,464	Pass
case2868 T4	40,163	9,488	19,073,056	Pass
case2868 T16	163,823	37,952	76,354,336	Pass
case2868 T48	493,583	113,856	229,507,104	Pass
case2868 T64	658,463	151,808	306,303,136	Pass
case2868 T96	988,223	227,712	460,334,496	Pass
case9241 T4	152,774	24,700	342,863,272	Pass
case9241 T6	230,376	37,050	514,308,838	Fail

TABLE V
MEDIAN SECONDS (IQR). CPU T6 IS ONE CENSORED CORRECTNESS
ATTEMPT, NOT A MEDIAN.

Case	HiGHS	CPU FP64	GPU FP64
1354/T4	1.463 (.015)	13.190 (1.571)	1.013 (.010)
1354/T16	7.265 (.005)	47.399 (3.906)	2.924 (.097)
1354/T48	32.095 (1.651)	153.046 (1.680)	9.481 (.048)
1354/T96	101.242 (.176)	336.657 (7.599)	21.084 (.274)
2868/T4	5.366 (.026)	60.350 (.253)	3.939 (.098)
2868/T16	22.138 (.219)	250.601 (1.645)	15.408 (.006)
2868/T48	76.239 (4.411)	804.863 (3.749)	49.968 (.027)
2868/T64	108.232 (.598)	1,078.892 (38.376)	68.383 (.008)
2868/T96	163.593 (1.394)	1,621.905 (19.300)	105.022 (.032)
9241/T4	142.479 (.376)	3,087.218 (13.350)	193.632 (.120)
9241/T6	963.957 (1.420)	3,600.093*	357.544 (.206)

TABLE VI
PREALLOCATION OUTCOMES FOR THE LARGEST RECONSTRUCTED CASE.

Row	Stop	Projected GiB	Boundary evidence
T16	Memory	94.435	Live budgets are 65.784/65.496 GiB.
T24	Index	141.663	Plan 2.532B > int32; exact 2.058B < int32.
T32	Index	188.897	Plan 3.376B and exact 2.744B > int32.

D. Timing decomposition and dispersion

Five repeats were planned; high-variability tracks were escalated to nine without deleting earlier samples. Table V reports medians and IQRs, while Fig. 1 overlays medians and full measured ranges on one shared logarithmic axis.

No speedup is claimed from Fig. 1. The shared-axis overlay makes the magnitude and scaling patterns visible without making the clocks commensurate. The elevated CPU maxima on several nine-repeat tracks show why a median alone is insufficient. The five GPU samples are tightly clustered on the large rows, but this is local repeatability evidence, not an A100 comparison.

The released CSV retains minima, maxima, standard deviations, repeat counts, and every underlying sample.

E. Resource boundaries

DGX Spark has 128 GB LPDDR5x unified memory [12]. The run-time system reported 130,663,165,952 bytes (121.690 GiB). The safety guard did not allocate against that nominal total. It required the projected host assembly plus device plan to fit within both 80% of observed host available pages and 80% of observed CUDA-free bytes.

The maximum cumulative process-lifetime host high-water mark was 95.375 GiB at T6; it is not an isolated

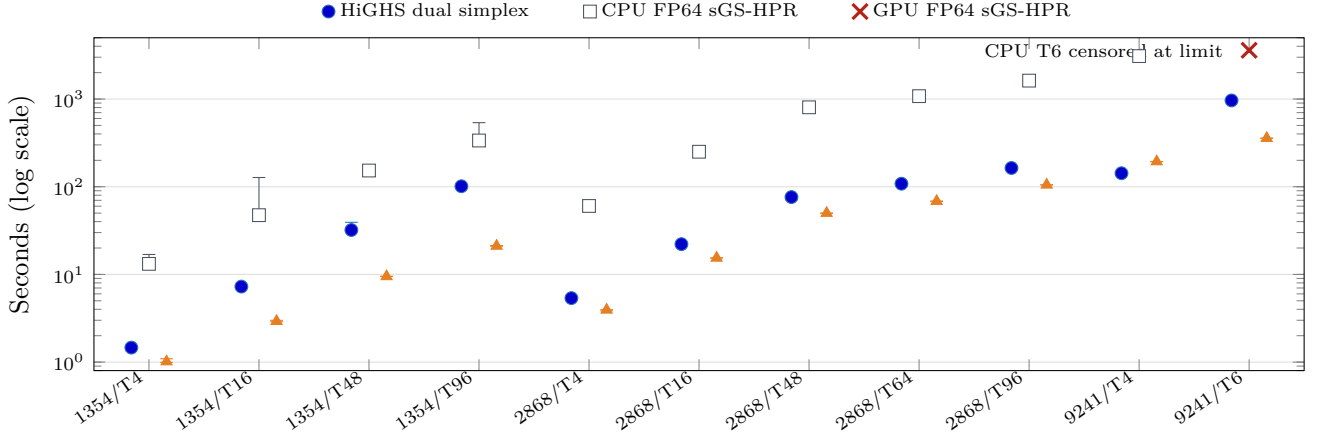


Fig. 1. Combined local timing evidence on one shared logarithmic axis. Each method is horizontally offset within a reconstruction; markers are medians and whiskers span measured minima and maxima. CPU T6 is one censored correctness attempt. The overlay aids visual comparison but does not establish a speedup because solver boundaries and algorithms differ.

solve peak. CUDA memory observations are snapshots, not per-solve high-water marks.

V. DISCUSSION

A. Reproduction boundary

The canonical signs, variables, constraint families, all 18 (m, n) pairs, update order, Halpern weights, tolerance, Ruiz/Pock–Chambolle preprocessing, initial σ , policy cadence, FP64 CSR path, and requested ALG2 selection are reproduced. CPU/GPU trajectories and all accepted original-space checks agree.

Author-instance identity is not reproduced. Placement, time series, reserve and ramp edits, storage parameters, costs, PTDF reference/sparsification, control code, source revision, precision details, and timer boundary are missing. Equation (55) and Table II imply $m_1 = T + N_{ESS}$ while Appendix A prose implies $T(1 + N_{ESS})$; the rank-one sign is inconsistent with the Schur complement; and the security-constrained label lacks an explicit outage model.

B. Limitations and next steps

The major uncertainties are instance construction and external performance, not whether the local code can execute. Timing cannot disentangle algorithm, support, implementation, hardware, runtime, or stopwatch effects. The T6 CPU track is censored and gives no median. T16/T24/T32 give safe boundary decisions, not solve times. Memory APIs limit peak attribution. The adaptive policy is a pinned HPR-LP transfer rather than unpublished DCOPF author code.

The most valuable follow-up is author-instance reconciliation: obtain modified cases, profiles, placements, parameters, construction code, precision, control logic, and timer rules; first match (m, n, nnz) , then objective/KKT fingerprints, then an equivalent A100 boundary. If those artifacts remain unavailable, 64-bit sparse indices, partitioned operators, or lower-peak assembly would be a new scalability study rather than a correction to this result.

VI. CONCLUSION

This study reconstructs the path from canonical DCOPF to a resident FP64 GPU sGS-HPR solver and corrects a quantitatively material rank-one sign error. CPU/GPU trajectories and original-unit constraints agree across the accepted public cases, but every reconstructed sparse support differs from the source paper and one required CPU solve is censored at its time limit. The evidence therefore establishes structural reproduction, not author-instance identity or controlled performance reproduction.

APPENDIX A

DETERMINISTIC PUBLIC-INSTANCE CONSTRUCTION

APPENDIX B

SOLVER IMPLEMENTATIONS AND MEASUREMENT BOUNDARIES

HiGHS reference. SciPy/HiGHS supplies the independent objective via `linprog(method='highs-ds')` with presolve, 10^{-9} primal and dual feasibility tolerances, and a 3,600-s limit [8], [9]. Its reported core time is the `linprog` call, including interface/model setup and the solve.

CPU oracle. The CPU sGS-HPR implementation uses SciPy sparse operators in FP64, a zero initial state, both equality sweeps, original-coordinate recovery, and independent residual and physical checks. Its measured time begins after model, preconditioner, and workspace preparation and includes the iterative solve and scheduled residual work.

GPU implementation. The target path uses CuPy 14.1.1 arrays, FP64 CSR matrices, resident state, and reused workspaces [10]. A low-level descriptor records `CUSPARSE_SPMV_CSR_ALG2`; normal and transpose probes verify the selected path. NVIDIA documents CSR ALG2 as deterministic for non-transpose SpMV [11]. Transfer ledgers reject unexpected full-state copies inside the iteration loop. The GPU clock uses the same post-preparation iterative/residual boundary as the CPU path.

TABLE VII
FROZEN PUBLIC-INSTANCE CONSTRUCTION RECIPE (SEED 20260803).

Component	Frozen rule
Networks	Versioned MATPOWER 8.1 snapshots of <code>case1354pegase</code> , <code>case2868rte</code> , and <code>case9241pegase</code> .
Load	Flat profile; active withdrawal is MATPOWER $P_d + G_s$ each hour.
Generators	Retain every row; originally offline rows are fixed to zero output and reserve; ramp limit is $0.1P_{\max}/h$.
Renewables	Cycle through generator buses in generator-row order; aggregate nameplate is 10% of base load, equally divided; availability 1; minimum 0.
Storage	Every second renewable location, cycling as needed; aggregate power 1% of base load, equal division; 4-h duration; initial state 50%; $\eta^{ch} = \eta^{dc} = 0.95$.
Reserve	Up and down requirements are each 1% of base load.
Costs	MATPOWER linear and constant polynomial terms; reject nonzero higher-order terms; renewable and storage-loss penalties are \$1/MWh.
Branches	Preserve positive rateA; zero rateA receives an outward-rounded finite bound proven redundant over the variable box; inactive rows are kept as zero-flow count rows but excluded from topology.
PTDF	Public type-3 reference bus; RHS chunks of 128; zero tolerance 10^{-12} ; phase-shift affine offsets retained; angle limits omitted because they are absent from the source model.
Time	One-hour intervals; no timing-driven tuning after the contract freeze.

CUDA initialization, model construction, preprocessing, first-run compilation, allocation, transfers, solver initialization, nested residuals, recovery, and complete runner wall time are separately named in the released timing ledger. Nested residual time is already part of loop time and is not added again.

APPENDIX C

PREPROCESSING, RESTART, AND PENALTY CONTROLS

The sGS multiplier metric for $\Delta y = (\Delta y_1, \Delta y_2)$ is

$$v = A_1^\top \Delta y_1 + A_2^\top \Delta y_2, \quad (17)$$

$$Q_y(\Delta y) = (A_1 v)^\top (A_1 A_1^\top)^{-1} (A_1 v) + \lambda \|\Delta y_2\|^2. \quad (18)$$

The restart merit is

$$\tilde{R}^2 = \|\Delta x\|^2 / \sigma + 2(A\Delta x)^\top \Delta y + \sigma Q_y(\Delta y). \quad (19)$$

Policy checks occur every 100 iterations; the first checkpoint forces a restart. Later checkpoints restart for sufficient decay $\tilde{R} \leq 0.2\tilde{R}_{ref}$, necessary decay with no local progress $\tilde{R} < 0.6\tilde{R}_{ref}$ and $\tilde{R} > \tilde{R}_{prev}$, or a long inner loop $k_{inner} \geq 0.2k_{total}$. An accepted restart copies the proximal point to anchor and current state and resets the inner counter.

With reference and candidate states,

$$\Delta x = \|x_c - x_r\|, \quad \Delta y = \sqrt{Q_y(y_c - y_r)}, \quad (20)$$

$$\sigma_{new} = \Delta x / \Delta y. \quad (21)$$

Both movements must lie strictly in $(10^{-16}, 10^{12})$, and the normalized dual/primal infeasibility ratio in $(10^{-8}, 10^8)$. Guard failure resets σ to 1. The policy source is HPR-LP commit `1941fbcfbf2dae14e4a439b22f0ea1e1c05f4a29` [2].

APPENDIX D

ALGEBRAIC STABILITY AND CONDITIONING

The positive-sum implementation of the reduced eigenvalue is

$$\gamma = G + R + \sum_s \frac{(\eta_s^{ch} - 1/\eta_s^{dc})^2}{(\eta_s^{ch})^2 + 1/(\eta_s^{dc})^2}, \quad (22)$$

which avoids subtracting nearly equal a and $T\alpha$. Nonpositive or scale-small γ is rejected. The resulting direct-oracle accuracy is reported with the other empirical results in Table III.

APPENDIX E

REPRODUCIBILITY, CODE, AND DATA AVAILABILITY

Code and data availability. Source, public case snapshots, configurations, immutable raw evidence, generated tables/figures, the machine-readable index, and this PDF are available at <https://github.com/thomas-digregorio/sGS-HPR-DC-SCOPF>. The report release is the Git tag `reproduction-paper-v6`. The public network source is MATPOWER 8.1, DOI 10.5281/zenodo.15871662 [3].

Missing source information. Exact reproduction would require the authors' modified cases, placements, profiles, reserve and ramp data, storage parameters, matrix construction, PTDF policy, precision, control code, source revision, and timer definition. Unknown values were not guessed or fitted to published timings.

Machine-readable evidence and regeneration. The tagged release includes a machine-readable evidence index, structural and benchmark ledgers, and timing summaries. Regeneration and validation commands are documented in the repository and do not rerun DGX allocations.

REFERENCES

- [1] Q. Wang, G. Zhang, Y. Yang, C. Ren, W. Wu, X. Zhao, M. Skoglund, and D. Sun, "An efficient GPU-based Halpern accelerating algorithm for large-scale DC optimal power flow," *IEEE Trans. Power Syst.*, vol. 41, no. 3, pp. 2187–2204, May 2026, doi: 10.1109/TPWRS.2025.3635652.
- [2] K. Chen, D. Sun, Y. Yuan, G. Zhang, and X. Zhao, "HPR-LP: An implementation of an HPR method for solving linear programming," *Math. Program. Comput.*, vol. 18, pp. 183–210, 2026, doi: 10.1007/s12532-025-00292-0.
- [3] R. D. Zimmerman and C. E. Murillo-Sánchez, *MATPOWER*, version 8.1, 2025, doi: 10.5281/zenodo.15871662; and R. D. Zimmerman, C. E. Murillo-Sánchez, and R. J. Thomas, "MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education," *IEEE Trans. Power Syst.*, vol. 26, no. 1, pp. 12–19, 2011.
- [4] C. Jozs, S. Fliscounakis, J. Maeght, and P. Panciatici, "AC power flow data in MATPOWER and QCQP format: iTesla, RTE snapshots, and PEGASE," arXiv:1603.01533, 2016.
- [5] S. Fliscounakis, P. Panciatici, F. Capitanescu, and L. Wehenkel, "Contingency ranking with respect to overloads in very large power systems taking into account uncertainty, preventive, and corrective actions," *IEEE Trans. Power Syst.*, vol. 28, no. 4, pp. 4909–4917, 2013, doi: 10.1109/TPWRS.2013.2251015.
- [6] D. Ruiz, "A scaling algorithm to equilibrate both rows and columns norms in matrices," Rutherford Appleton Laboratory, Tech. Rep. RAL-TR-2001-034, 2001.

- [7] T. Pock and A. Chambolle, “Diagonal preconditioning for first order primal-dual algorithms in convex optimization,” in *Proc. IEEE ICCV*, 2011, pp. 1762–1769, doi: 10.1109/ICCV.2011.6126441.
- [8] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.
- [9] Q. Huangfu and J. A. J. Hall, “Parallelizing the dual revised simplex method,” *Math. Program. Comput.*, vol. 10, pp. 119–142, 2018.
- [10] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, “CuPy: A NumPy-compatible library for NVIDIA GPU calculations,” in *Proc. Workshop on Machine Learning Systems, NeurIPS*, 2017.
- [11] NVIDIA Corp., “cuSPARSE library,” CUDA Toolkit Documentation, 2026. [Online]. Available: <https://docs.nvidia.com/cuda/cusparse/>
- [12] NVIDIA Corp., “DGX Spark User Guide: Hardware overview,” 2026. [Online]. Available: <https://docs.nvidia.com/dgx/dgx-spark/hardware.html>